Resumen teórico concurrente

¿Por qué es necesaria la Programación Concurrente?

- Aplicaciones con estructura más natural.
    - Más apropiado programar múltiples actividades independientes y concurrentes.
- Mejora en la respuesta.
    - No bloquear la aplicación completa por E/S.
    - Incremento en el rendimiento de la aplicación por mejor uso del hardware (ejecución paralela).
- Sistemas distribuidos
    - Una aplicación en varias máquinas.
    - Sistemas C/S (cliente/servidor) o P2P.

Objetivos de los sistemas concurrentes

Ajustar el modelo de arquitectura de hardware y software al problema del mundo real a resolver.
Incrementar la performance, mejorando los tiempos de respuesta de los sistemas de cómputo, a través de un enfoque diferente de la arquitectura física y lógica de las soluciones.
Algunas ventajas:

- La velocidad de ejecución que se puede alcanzar.
- Mejor utilización de la CPU de cada procesador.
- Explotación de la concurrencia inherente a la mayoría de los problemas reales.

Concurrencia vs. Paralelismo

- **Concurrencia:** Es un concepto de software que se refiere a la capacidad de ejecutar múltiples actividades en paralelo o simultáneamente. No está restringida a una arquitectura de hardware específica; puede darse incluso en un solo procesador mediante la intercalación de tiempo o multiprogramación.
- **Paralelismo:** Está asociado a la ejecución concurrente en múltiples procesadores reales de hardware, con el objetivo principal de reducir el tiempo de ejecución.

- La necesidad de programar concurrentemente surge porque el mundo real no es secuencial, mejora el tiempo de respuesta (por ejemplo, evitando que el sistema se bloquee por entrada/salida), y permite aprovechar los procesadores multicore actuales donde el límite físico de velocidad ya no permite hacer procesadores individuales más rápidos.

Comportamiento de los Procesos e Hilos

- **Proceso vs. Hilo (Thread):** Un proceso tiene su propio espacio de direcciones y recursos. En cambio, un hilo (proceso liviano) tiene su propio contador de programa y pila de ejecución, pero comparte el espacio de memoria y recursos con otros hilos del mismo proceso, lo que exige mecanismos para evitar interferencias.
- Los procesos concurrentes pueden relacionarse de tres formas: ser totalmente independientes (poco frecuente), competir por recursos (lo que puede generar problemas como interbloqueos o inanición), o cooperar para una tarea común (lo que requiere sincronización).

Comunicación y Sincronización

- **Comunicación:** Los procesos pueden comunicarse mediante **Memoria Compartida** (leyendo y escribiendo en el mismo espacio, lo que obliga a bloquear y liberar el acceso) o mediante **Pasaje de Mensajes** (transmitiendo información a través de canales físicos o lógicos con protocolos específicos).
- **Sincronización por Exclusión Mutua:** Se usa para asegurar que solo un proceso acceda a un recurso compartido (sección crítica) en un instante determinado.
- **Sincronización por Condición:** Se utiliza para bloquear la ejecución de un proceso hasta que se cumpla una condición específica, como por ejemplo esperar a que haya espacio libre en un buffer.

Desafíos y Problemas Clásicos

- **Interferencia:** Se da cuando un proceso realiza una acción que invalida las suposiciones o resultados esperados por otro proceso concurrente.
- **Deadlock (Interbloqueo):** Situación donde dos o más procesos se quedan trabados esperando que el otro libere un recurso. Para que exista, deben darse cuatro condiciones necesarias y suficientes:
    - **Recursos reusables serialmente:** Los procesos comparten recursos que pueden usar con exclusión mutua.
    - **Adquisición incremental:** Los procesos retienen los recursos que poseen a la vez que esperan recursos adicionales.
    - **No-preemption:** Una vez que los recursos son adquiridos por un proceso, los demás procesos deben esperar que se liberen voluntariamente.
    - **Espera cíclica:** Existe una cadena circular (ciclo) de procesos tal que cada uno tiene un recurso que su sucesor en el ciclo está esperando adquirir.
- **Existen 2 situaciones que deben evitarse:**
    - **Inanición:** Un proceso no logra acceder a recursos compartidos.
    - **Overloading:** La carga asignada excede la capacidad de procesamiento.

Arquitecturas de Hardware

- El material distingue entre multiprocesadores de **memoria compartida** (como los multicore, GPUs, o esquemas UMA/NUMA donde los problemas principales son la sincronización y la consistencia de caché) y arquitecturas de **memoria distribuida** (como multicomputadoras o clusters, donde la interacción es únicamente por pasaje de mensajes).

Acciones Atómicas a Nivel de Código

- **Granularidad y Atomicidad:** Una asignación simple como `X = X + 1` no es atómica por hardware (requiere cargar, sumar y guardar). Por lo tanto, el orden relativo (interleaving) en el que se

ejecutan estas sub-operaciones puede generar resultados inesperados.
- **Propiedad "A lo sumo una vez" (ASV):** Una instrucción parece y actúa de forma atómica si contiene como máximo una "referencia crítica" (una variable modificada por otro proceso). Si no se cumple esto, se deben usar sentencias explícitas de grano grueso como `<await (condición) sentencias;>` para garantizar la exclusión mutua.

Correctitud: Safety vs. Liveness

- **Seguridad (Safety):** Propiedades que garantizan que "nada malo ocurra", asegurando que los estados sean consistentes (ej. la exclusión mutua).
- **Vida (Liveness):** Propiedades que aseguran que "eventualmente ocurra algo bueno", como que un programa no tenga deadlocks o que termine exitosamente.
- **Fairness (Equidad):** Las políticas de planificación del sistema operativo deben intentar garantizar que todos los procesos tengan la oportunidad de avanzar (existen niveles de equidad incondicional, débil y fuerte).

Condiciones para sección crítica

1. **Exclusión Mutua (*Mutual Exclusion*) — Propiedad de Seguridad (*Safety*)**
    - **Definición:** A lo sumo un único proceso puede estar ejecutando dentro de su Sección Crítica en un instante de tiempo dado.
    - **Explicación:** Si el Proceso A está dentro de la SC, ningún otro proceso puede entrar hasta que A salga.
2. **Ausencia de Deadlock / Livelock (*Deadlock Freedom*) — Propiedad de Vida (*Liveness*)**
    - **Definición:** Si dos o más procesos están intentando entrar concurrentemente a sus Secciones Críticas, al menos uno debe tener éxito y lograr entrar.

    - **Explicación:** Los procesos no pueden trabarse mutuamente entre sí en el protocolo de entrada de forma que ninguno avance jamás.
3. **Ausencia de Demora Innecesaria (*Absence of Unnecessary Delay*) — Propiedad de Seguridad (*Safety*)**
    - **Definición:** Si un proceso quiere entrar a su Sección Crítica y el resto de los procesos están en su Sección No Crítica (SNC) o ya terminaron, el primer proceso no debe ser impedido de entrar inmediatamente.
    - **Explicación:** Un proceso que no compite por el recurso ni está en la SC no puede bloquear a otro que sí quiere usarlo. No puede haber dependencias innecesarias de procesos inactivos.
4. **Eventual Entrada (*Eventual Entry / Starvation Freedom*) — Propiedad de Vida (*Liveness*)**
    - **Definición:** Todo proceso que intente entrar a su Sección Crítica debe tener la garantía de que eventualmente logrará hacerlo (ausencia de inanición o *starvation*).
    - **Explicación:** Ningún proceso puede quedarse esperando afuera indefinidamente mientras otros procesos entran y salen repetidas veces de la SC. Esta condición suele depender de que la política de planificación (*scheduling*) sea equitativa (*fair*).

Algoritmos Clásicos de Solución (Busy Waiting / Spin Locks)

Problemas del Busy Waiting: Es ineficiente en sistemas multiprogramados (desperdicia ciclos de CPU chequeando una variable) y genera alta contención de memoria (*memory contention*) al consultar continuamente el mismo bus.

A. Instrucciones Especiales de Hardware (Grano Fino)

- **Definición:** Para evitar usar sentencias abstractas como `<await>`, los procesadores proveen instrucciones atómicas reales como `Test & Set (TS)` o `Fetch & Add (FA)`.
- **Spin Lock Básico (con TS):** Un `while (TS(lock)) skip;` que intenta ganar el lock.

- **Problema:** Alta contención de memoria al intentar escribir continuamente.
- **Solución (Mejora):** *Test-and-Set*. Se lee el lock pasivamente y solo se hace el `TS` atómico cuando parece estar libre (`while (lock) skip; while (TS(lock))...`).

B. Algoritmo Tie-Breaker (Desempate)

- **Definición:** Solución pura por software que no requiere instrucciones especiales de hardware (solo requiere *scheduling* débilmente fair).
- **Mecanismo:** Usa variables para indicar intención (`in1`, `in2`) y una variable de desempate (`ultimo`). El último proceso en declarar que quiere entrar es el que "cede el paso" y se bloquea.
- **Problema:** Escalarlo a *N* procesos es muy complejo, ya que requiere armar un bucle de *N-1* etapas compitiendo de a pares.

C. Algoritmo Ticket

- **Definición:** Funciona como la cola del supermercado.
- **Mecanismo:** Hay un número global (`numero`) que se entrega al que llega, y un número que indica a quién están atendiendo (`proximo`). El proceso usa `FA(numero, 1)` para sacar número y luego espera con `while (turno[i] != proximo) skip;`.
- **Problemas:** Las variables globales crecen infinitamente (deben resetearse en la práctica) y depende de que exista la instrucción de hardware `Fetch-and-Add` para sacar número atómicamente.

D. Algoritmo Bakery (Panadería de Lamport)

- **Definición:** Algoritmo puro de software que es 100% *fair* y no necesita hardware especial ni contadores globales.
- **Mecanismo:** Cada proceso revisa los números que tienen los demás y se autoasigna el máximo + 1. Luego espera su turno comparando su número con el resto (pasa el de número menor).
- **Problema:** Costoso computacionalmente, ya que para calcular su número tiene que iterar sobre los valores de todos los demás procesos.

Sincronización Barrier (Barreras)

Definición: Un punto en el código donde todos los procesos de un grupo deben llegar y detenerse. Ninguno puede avanzar más allá de la barrera hasta que **todos** hayan llegado a ese punto. Son muy útiles en algoritmos iterativos paralelos.

Modelos de Implementación de Barreras

1. **Contador Compartido:** Usa una variable global incrementada por `FA`. *Problema:* Contención de memoria al todos acceder a la misma variable, y dificultad para reiniciar el contador para la siguiente iteración iterativa.
2. **Flags y Coordinador (Esclavo-Maestro):** Los *Workers* (esclavos) avisan que llegaron en un arreglo de `arribos` y se frenan. Un proceso extra (*Coordinador*) lee el arreglo y, cuando todos llegaron, baja la barrera mediante un arreglo `continuar`. *Problema:* Requiere un proceso y procesador adicional, y su tiempo de ejecución escala linealmente con la cantidad de procesos.
3. **Árboles (Combining Tree):** Los procesos se organizan en forma de árbol. Las señales de que llegaron suben por las ramas, y cuando llega a la raíz, la señal de continuar baja. Es más eficiente para muchos procesos.
4. **Barreras Simétricas (Butterfly Barrier):** No hay roles distintos. Todos son coordinadores a la vez. En cada etapa, los procesos se sincronizan en pares cruzados a diferentes "distancias" (potencias de 2). Toma **log2(N)** etapas.

Semáforos

¿Qué es un semáforo?

Es un tipo de dato abstracto que encapsula un contador entero (no negativo) y solo permite dos operaciones atómicas:

- **`P(s)` (Wait / Demorar):** Espera a que el semáforo `s > 0` y luego lo decrementa (`s = s - 1`). Se usa para *pedir* permiso.
- **`V(s)` (Signal / Señalar):** Incrementa el semáforo (`s = s + 1`). Se usa para *avisar* o liberar un recurso.

Tipos

- **Semáforo General (o Counting):** Puede tomar cualquier valor positivo (ej. para contar 5 impresoras libres).

- **Semáforo Binario:** Sólo vale 0 o 1 (sirve como un "candado" o flag de estado).

¿Por qué usar semáforos?

Hasta ahora, las soluciones al problema de la Sección Crítica (como el Algoritmo Ticket o Bakery) se basaban en **Busy Waiting** (espera activa o *spinning*).

- **El problema:** Un proceso bloqueado se queda girando en un `while (condición)` gastando ciclos de CPU sin hacer nada útil. En sistemas multiprogramados, esto es súper ineficiente.
- **La solución:** Los **semáforos**. Permiten bloquear un proceso "poniéndolo a dormir" en una cola hasta que ocurra un evento, liberando el procesador para que otro trabaje.

¿Cuándo usar qué técnica de Semáforos?

- **Exclusión Mutua (Proteger Secciones Críticas)**
  - **Cuándo usarlo:** Cuando necesitás que solo un proceso acceda a un recurso compartido (ej. modificar una variable global).
- **Señalización de Eventos y Barreras**
  - **Cuándo usarlo:** Cuando un proceso A debe esperar a que el proceso B termine una tarea antes de continuar, o cuando *N* procesos deben esperar en un punto hasta que todos lleguen (Barrera).
- **Contadores de Recursos (Productor/Consumidor)**
  - **Cuándo usarlo:** Cuando tenés un recurso con múltiples unidades idénticas disponibles (ej. un buffer con 10 lugares libres).
- **Semáforos Binarios Divididos**
  - **Cuándo usarlo:** En problemas complejos de sincronización por condición (como Múltiples Productores/Consumidores, o Lectores/Escritores) donde distintos grupos de procesos esperan por condiciones diferentes, pero necesitan garantizar exclusión mutua global.
- **Técnica "Passing the Baton"**
  - **Cuándo usarlo:** Cuando tenés reglas de prioridad complejas que no se resuelven con un `P()` directo. Por

ejemplo, en "Lectores/Escritores", donde querés controlar exactamente a quién despertar según quién está esperando.
    - **Concepto:** Usa semáforos binarios divididos. Un proceso entra a la SC (toma el "testimonio"). Cuando está por salir, en vez de liberar el lock general, revisa variables de estado y decide hacerle un `V()` (despertar) directamente al semáforo privado del proceso específico al que quiere pasarle el turno.
- **Alocación de Recursos y Scheduling (Ej: Shortest-Job-Next)**
    - **Cuándo usarlo:** Cuando los procesos que esperan no deben ser atendidos en orden de llegada, sino por otra política (ej. el trabajo más corto primero, SJN).

Monitores

¿Qué es un Monitor?

Un **Monitor** es un mecanismo de abstracción de datos para programación concurrente. Nace para solucionar los problemas típicos del uso de semáforos, donde el control de acceso a recursos compartidos está disperso por todo el código, dificultando la verificación y mantenimiento.

Sus características principales son:

- **Abstracción y Encapsulamiento:** Agrupa (encapsula) las variables que representan el estado del recurso compartido y los procedimientos (*procedures*) que implementan las operaciones permitidas sobre él.
- **Interfaz Clara:** Para los procesos externos, el monitor actúa como un objeto pasivo. Solo pueden interactuar con él invocando los nombres de los procedimientos definidos en su interfaz (ej. `NombreMonitor.procedure()`). Un proceso no necesita saber cómo está implementado internamente.
- **Exclusión Mutua Implícita:** La mayor ventaja del monitor. Garantiza automáticamente que *solo un proceso a la vez* puede estar ejecutando un procedimiento dentro del monitor. No hace

falta programar explícitamente operaciones de bloqueo (como los P() o V() de semáforos) para proteger la sección crítica.

Sincronización por Condición en Monitores

Mientras que la exclusión mutua la maneja el monitor automáticamente, la **Sincronización por Condición** se debe programar de forma explícita utilizando **Variables Condición** (del tipo `cond`).

Una variable condición representa internamente una cola de procesos demorados. Sobre estas variables solo se pueden aplicar ciertas operaciones:

- **wait(cv):** El proceso que ejecuta esta instrucción se bloquea, se encola al final de la cola cv y **libera el acceso exclusivo al monitor** para que otro proceso pueda entrar.
- **signal(cv):** Despierta al proceso que está primero en la cola de la variable cv y lo saca de ella. Si la cola está vacía, la instrucción no tiene efecto.
- **signal_all(cv):** Despierta a todos los procesos que estén esperando en la cola de cv, vaciándola completamente.

Nota:

- *Aunque no se usan en la práctica, existen operaciones adicionales como empty(cv) para verificar si la cola está vacía, y wait(cv, rank) para demorar procesos priorizados por un rango*.
- Cond tiene una cola que **siempre** usa orden FIFO.

Disciplinas de Señalización

Cuando un proceso A ejecuta un `signal()` despertando a un proceso B, ¿qué ocurre en ese instante dado que solo uno puede estar activo en el monitor? Existen dos disciplinas principales:

- **Signal and Continue:** El proceso que emite el signal() (Proceso A) **continúa su ejecución** dentro del monitor. El proceso despertado (Proceso B) **pasa a un estado de competencia**, esperando que el monitor se libere para poder entrar y retomar su ejecución justo en la instrucción siguiente al wait().

- **Signal and Wait:** El proceso que emite el signal() (Proceso A) sede inmediatamente el control y se bloquea compitiendo para reingresar después. El proceso despertado (Proceso B) entra a ejecutar al monitor de forma inmediata.

Nota: Signal and wait no se utiliza en la práctica(igualmente puede entrar en el exámen teórico).

Técnicas y Patrones de Diseño comunes

A la hora de programar con monitores, se suelen aplicar distintas técnicas para resolver problemas clásicos:

- **Sincronización por Condición Básica:** Usar ciclos while junto con wait() para reevaluar la condición al despertar (esencial con "Signal and Continue" para evitar que el estado haya cambiado desde que el proceso fue despertado hasta que logró entrar al monitor).
- **Passing the Condition:** Técnica usada para delegar el control explícitamente a un proceso demorado sin alterar contadores globales (muy útil en simulaciones de semáforos o Lectores y Escritores).
- **Rendezvous (Encuentro):** Barrera de sincronización en dos o más etapas donde distintas partes (ej. Peluquero y Cliente) deben llegar a un punto común antes de poder continuar.
- **Variables Condición Privadas:** Se usa un arreglo de variables condición (ej. cond espera[N]) para despertar procesos específicos identificados por un ID, manejando el orden explícitamente a través de colas ordenadas internas.

PMS: solo teórico

RPC Y Rendezvous: son bidireccionales(sincronico OBVIO)

Cr es equivalente a **fin de línea** (**EOL** es utilizado para marcar el fin de línea)

EOS es referido como centinela.

Memoria Distribuida

Conceptos Generales

- **Arquitecturas de memoria distribuida:** Sistemas formados por múltiples procesadores, donde cada uno tiene su propia memoria local. Estos procesadores están interconectados por una red de comunicaciones.
- **Programa distribuido:** Es un programa concurrente donde los procesos interactúan y se sincronizan exclusivamente mediante el intercambio de mensajes, ya que **no comparten variables en memoria**.
- **Primitivas de pasaje de mensajes:** Conforman la interfaz con el sistema de comunicaciones. Actúan simultáneamente como mecanismo de transferencia de datos y de sincronización (similar a los semáforos, pero con carga útil).
- **Exclusión Mutua:** En estos sistemas, la exclusión mutua no requiere de un mecanismo especial (como P()/V() o Monitores), porque las variables son estrictamente locales a cada proceso.

Canales

- **Definición:** Los canales son el único elemento que los procesos comparten. Funcionan como **colas de mensajes enviados y aún no recibidos**.
- **Características del acceso:** El acceso a los canales es atómico y respeta estrictamente el orden de llegada (FIFO).
- **Capacidad:** Teóricamente, los canales en Pasaje de Mensajes Asincrónicos (PMA) se consideran ilimitados, aunque en la práctica dependen del buffer del sistema operativo. Se asume que los mensajes ni se pierden ni se modifican.
- **Clasificación de canales según su uso:**
    - **Mailbox (Buzón):** Cualquier proceso puede enviar mensajes a este canal y cualquier proceso puede recibirlos.

    - **Input port (Puerto de entrada):** Múltiples procesos pueden enviar mensajes a este canal, pero hay un único proceso receptor autorizado a leerlos.
    - **Link (Enlace):** Canal exclusivo entre dos procesos; tiene un único emisor y un único receptor.

Primitivas de Pasaje de Mensajes Asincrónicos (PMA)

- **send (Emisión):** Agrega un mensaje al final de la cola del canal. En PMA, el send **no es bloqueante**; el proceso emisor coloca el mensaje y continúa su ejecución inmediatamente.
- **receive (Recepción):** Primitiva que **demora (bloquea) al proceso receptor** hasta que haya al menos un mensaje en la cola del canal. Cuando hay un mensaje, toma el primero (FIFO) y lo guarda en variables locales. Al ser bloqueante, no es necesario hacer espera activa (*polling*).
- **empty(ch):** Función que determina si la cola de un canal está vacía. Debe usarse con cuidado, ya que su evaluación puede quedar obsoleta rápidamente en entornos concurrentes.

Notas:

- Send es atómica.
- Receive toma un único mensaje.

Patrones de Interacción

Los mecanismos de sincronización se adaptan mejor a diferentes formas en que los procesos interactúan:

- **Productores y Consumidores (Filtros o Pipes):** Un proceso (filtro) recibe mensajes de canales de entrada, los procesa y envía el resultado a canales de salida. Su salida es función de su estado inicial y de los valores recibidos (ej. Redes de ordenación).
- **Clientes y Servidores:** Un proceso servidor maneja pedidos de múltiples clientes. Los clientes envían un mensaje por un canal de requerimiento general y esperan la respuesta en un canal propio. Muestra una gran dualidad con los monitores, ya que el servidor encapsula las variables y maneja operaciones en un bucle.

- **Pares Interactuantes (Peers):** Procesos de jerarquía similar que necesitan intercambiar valores. Las topologías típicas de solución son:
    - **Centralizada:** Todos envían a un nodo central que procesa y reenvía.
    - **Simétrica (Full connected):** Todos se comunican con todos.
    - **Anillo:** Cada proceso envía solo a su sucesor y recibe de su predecesor.

PMS Y CSP

En Pasaje de Mensajes Sincrónicos (**PMS**) el emisor se queda bloqueado hasta que el receptor recibe el mensaje. Esa es la única diferencia con **PMA.**

¿Qué es PMS?

Los canales son de tipo link o punto a punto: 1 emisor y 1 receptor. El **receive** es igual que en PMA, lo que cambia es el envío.

- **send** (PMA): Deja el mensaje en la cola y sigue.
- **syc_send** (PMS): Deja el mensaje y espera que el receptor lo tome.

Como el emisor no puede avanzar hasta que lo reciban, nunca hay más de un mensaje pendiente por canal. Por lo tanto PMS usa menos memoria, pero el costo es la concurrencia, mientras el emisor espera no hace nada útil.

Desventajas de PMS

- **Menos concurrencia:** Cada par send/receive se completa al ritmo del proceso más lento.
    - **Con PMS:** En cada mitad, el rápido espera al lento. Si la relación es 10 a 1, el tiempo total se multiplica por 10.

    - **Con PMA:** En la primera mitad el productor encola mensajes sin esperar. En la segunda, el consumidor “descuenta” tiempo consumiendo esa cola.

Para lograr el efecto de PMA usando PMS hay que interponer un proceso buffer entre los dos.

- **Más probabilidad de deadlock:** Todo sync_send necesita un receive que le haga matching en el otro proceso, y en el orden correcto. Un caso típico es el intercambio de valores entre dos procesos.

```
chan in1(int), in2(int);

Process P1
{ int valorA = 1, valorB;
  sync_send in2(valorA);   # se bloquea esperando que P2 reciba
  receive in1(valorB);
}

Process P2
{ int valorA, valorB = 2;
  sync_send in1(valorB);   # se bloquea esperando que P1 reciba
  receive in2(valorA);
}
```

P1 espera que P2 haga receive in2, y P2 espera que P1 haga receive in1. Ninguno llega a su receive. Con PMA este mismo código funciona, porque los send no bloquean.

Versión correcta: Se rompe la simetría, P2 recibe primero.

```
Process P2
{ int valorA, valorB = 2;
  receive in2(valorA);
  sync_send in1(valorB);
}
```

La regla práctica en PMS, **si un proceso arranca enviando, el otro tiene que arrancar recibiendo.**

CSP: El lenguaje para PMS

CSP introdujo 2 ideas: PMS y comunicación guardada (pasaje de mensaje con waiting selectivo). OCCAM, ADA y SR se basan en CSP.

Canales y sentencias

En CSP no se declaran canales globales tipo mailbox. El canal es un link directo entre dos procesos, half-duplex y nominado: Cada proceso nombra al otro

Sentencia	Símbolo	Forma general	Se lee
Salida	`!` (shriek, bang)	`Destino ! port(e1, ..., en);`	"le envío a Destino"
Entrada	`?` (query)	`Fuente ? port(x1, ..., xn);`	"recibo de Fuente"

- **Destino** y **fuente** nombrar un proceso simple o un elemento de un arreglo de procesos.
- **Fuente[*]** permite recibir de cualquier elemento del arreglo.Solo vale para la entrada.
- **port** es una etiqueta para distinguir clases de mensajes. Se puede omitir si hay una sola

Comunicación guardada

Resuelve este problema: ? y ! son bloqueantes, y un proceso que se comunica con varios otros no sabe en qué orden van a querer hacerlo. Si elige mal el orden, se queda bloqueado en uno mientras otro lo está esperando.

La solución es ofrecer varias comunicaciones a la vez y ejecutar la que esté lista. Se usan los comandos guardados de Dijkstra, con una sentencia de comunicación dentro de la guarda. **B; C -> S;**

- **B:** Condición booleana. Se puede omitir y se asume true.
- **C:** Sentencia de comunicación (? o !).
- **B y C juntas:** Forman la guarda.
- **S:** Lo que se ejecuta después de la comunicación.

Tres estados de una guarda

Estado	Cuándo	Qué significa
Éxito	B es true y C se puede ejecutar sin demora	Hay alguien del otro lado listo para hacer matching
Falla	B es falsa	Esa alternativa no se considera; C ni se mira
Se bloquea	B es true pero C no puede ejecutarse todavía	Hay que esperar a que el otro proceso llegue a su sentencia

Fallar depende solo de B. Bloquearse depende de C.

Como se ejecuta un if

```
if B1; comunicación1 → S1;
□  B2; comunicación2 → S2;
fi
```

1. Se evalúan las guardas.
    - Si **todas fallan**, el if termina sin efecto.
    - Si **al menos una tiene éxito**, se elige una de ella en forma no determinística.
    - Si **algunas se bloquean** y ninguna tiene éxito, se espera hasta que alguna tenga éxito.
2. Se ejecuta la sentencia de comunicación de la guarda elegida.
3. Se ejecuta la sentencia if correspondiente.

El do funciona igual, pero repite hasta que todas las guardas fallen.

RPC, Rendezvouz y ADA

RPC y Rendezvouz son mecanismos pensados para cliente/servidor: el cliente hace un *call* y queda demorado hasta recibir el resultado, como si llamara a un procedimiento. La diferencia está en quien atiende. En RPC un proceso **nuevo** por cada llamado, en Rendezvous un proceso **existente**, que decide cuándo aceptar.

RPC y Rendezvouz

- La comunicación es bidireccional, así que hacen falta dos tipos de canales (requerimientos y respuestas).
- Cada cliente necesita un canal de respuesta distinto.

	RPC	Rendezvous
Quién atiende el llamado	Un proceso nuevo (al menos conceptualmente) por cada llamado	Un proceso ya existente
Cómo se declara la operación	Un `proc` por operación	Una sentencia de entrada (`in` o `accept`)
Atención de llamados	Concurrente	De a uno por vez
Servidor	Pasivo: un conjunto de procedimientos	Activo: decide cuándo y qué aceptar
Sincronización	Hay que programarla dentro del módulo	Viene incluida en el mecanismo
Ejemplo de lenguaje	Java (RMI)	Ada

RPC

Los programas se descomponen en módulos, que tienen procesos y procedures, estos pueden residir en espacios de direcciones distintos.

- Los procesos de un módulo pueden compartir variables y llamar a procedures de ese módulo.
- Un proceso solo puede comunicarse con otro módulo invocando los procedimientos que ese módulo exporta.
- Los procesos locales se llaman **background**, para distinguirlos de las operaciones exportadas.

- **Sincronización de módulos**

  Por sí mismo, RPC es solo un mecanismo de comunicación. La sincronización entre el llamador y su server es implícita, pero los procesos dentro de un módulo también tienen que sincronizarse entre sí.

  Si los procesos de un módulo se ejecutan con:

  - **Exclusión mutua:** Las variables compartidas quedan protegidas, solo hay que programar la sincronización por condición.

- **Concurrentemente:** Hay que programar la exclusión mutua y sincronización por condición, con cualquier método(semáforos, monitores, rendezvous).

Generalmente se asume que se ejecutan **concurrentemente.**

Rendezvous

Problemas de RPC que motiva el rendezvous

- Solo da comunicación intermódulo. La sincronización hay que programarla a parte.
- A veces hacen falta procesos extra solo para manipular los datos comunicados.

Rendezvous combina **comunicación y sincronización.**

- El cliente invoca con *call*, igual que RPC.
- La operación la sirve un proceso existente, con una **sentencia de entrada.**
- Las operaciones se atienden **de una a la vez**.

ADA

- Un programa tiene **tasks** que se ejecutan de manera independiente.
- Los puntos de invocación de una tarea son los **entrys**, declarados en la parte visible.
- Una parte decide si acepta una comunicación con la primitiva **accept.**
- Se puede declarar un **type task** y crear varias instancias (arreglos, puntero, instancia simple).

  Tipos de entry call

  - **Incondicional(simple):** Se demora hasta que termina la operación (Tarea.entry(params);)

- **Condicional(else):** Si el servidor no puede aceptarlo inmediatamente, ejecuta la rama else y no queda esperando.
- **Temporal:** Espera a ser atendido como máximo cierta cantidad de segundos. Si no lo acepta en ese tiempo, desiste y ejecuta las sentencias del **delay.**

Diferencias con rendezvous general

- *When* **no puede mirar los parámetros** del llamado. Se evalúa antes de aceptar.
- No hay expresión de scheduling. Los parámetros a un entry se atienden en orden de llegada.